7주차 같은 피처를 두 번 만들지 않으려면

> **7주차 · 피처 스토어**

□ 1주차에 데이터 엔지니어링이 만든 값을 MLOps가 그대로 모델 입력으로 쓰는 것이 피처(feature)이고, 바로 이 피처에서 학습-서빙 불일치가 생긴다고 예고했음
  ○ 그때 `pm10_bad`라는 같은 이름의 피처를 훈련 경로와 서빙 경로에서 서로 다른 규칙으로 계산해 봤고, 측정소 39곳 중 11곳에서 값이 갈렸음. 두 경로 모두 오류 없이 끝나 어긋난 흔적이 어디에도 남지 않았음
  ○ 그래서 해결은 임계값을 맞추는 것이 아니라 피처 정의를 한곳에 두는 것이라고만 적어 두고, 그 일을 맡는 도구는 7주차로 넘겼음
  ○ 이번 주에 그 도구를 직접 만듦. 6주차에 배치로 확정한 일별 집계를 모델이 정해진 규격으로 읽게 만드는 저장소, 곧 피처 스토어(feature store)를 최소 구성으로 만듦

1. 오늘의 질문

□ 훈련 코드와 서빙 코드가 각자 "전일 민원 건수"를 계산하면 무엇이 어긋나는가?
□ 같은 피처를 왜 두 곳에 나누어 두고, 두 곳의 값이 같다는 것은 무엇으로 보이는가?
□ 7월 2일 오전 9시의 훈련 샘플에 그날 하루치 집계를 붙이면 왜 안 되는가?
□ 이 피처를 누가 책임지고, 그 값이 언제 알려진 것인지는 어디에 적어 두는가?

2. 이 장에서 실행할 것

□ 6주차까지 배치로 확정한 일별 집계를 모델이 입력으로 쓰려면, 그 계산이 한 곳에만 있어야 함
  ○ 지금은 한 곳에 없음 — 훈련 파이프라인과 서빙 API가 각자 "전일 민원 건수"를 계산하고, 같은 날에 서로 다른 값이 나옴
  ○ 이 장은 그 계산을 한 곳에 모으고, 모아 둔 값을 훈련과 서빙이 같은 정의로 읽게 만드는 네 가지를 다룸

이 장의 개념	무엇을 막는가	다루는 절
피처를 한 번 정의하고 훈련과 서빙이 같은 정의를 읽음	같은 이름의 피처를 두 곳에서 따로 계산해 값이 갈림	3
offline store(이력 전체) / online store(키별 최신값)	대량 이력 조회와 단건 최신값 조회를 한 저장소로 처리하려다 양쪽 다 느려짐	4
시점 조인 — 기준 시점 이전에 알려진 값만 붙임	그 시점에 아직 확정되지 않은 값이 훈련 샘플에 들어감(미래 누출)	5
피처 정의서의 소유자·원천·타임스탬프 의미	원천 부서가 집계 방식을 바꾼 사실을 소비하는 모델 쪽이 모름	6

표 7.1 이 장의 네 개념과 각각이 막는 어긋남

□ 네 가지가 한 장에 모인 이유를 한 줄로 밝힘

- ○ 피처는 한 번 정의해 두면 여러 모델이 같은 값을 동시에 읽는 자산이므로, 값 하나가 어긋나면 그 피처를 쓰는 모델이 함께 어긋남
- ○ 그래서 각 모델이 값을 따로 확인하는 대신, **저장한 두 곳의 값이 같은지를 한 곳에서 대조**함(4절)

□ 이 장에서 실행할 코드는 넷이며 순서가 정해져 있음

순서	실행할 파일	하는 일	절	선행 조건
1	`7-2-count-skew.py`	정제 단계별로 같은 날 건수를 세어 대조	3	없음
2	`7-3-materialize-consistency.py`	온라인 적재와 훈련-서빙 일관성 점검	4	없음
3	`7-4-point-in-time-join.py`	시점 조인으로 훈련 샘플에 피처 붙이기	5	없음
4	`7-1-feature-store-minimal.py`	같은 흐름을 Feast 엔진으로 실행	6	feast·pandas·pyarrow 설치

□ 1~3번은 파이썬만 있으면 돌아감. 표준 라이브러리만 쓰고 난수도 외부 접속도 없음

□ 4번만 패키지 설치가 필요하며, 설치는 아래 한 줄로 함

```
python scripts/setup_practice.py 7   # 저장소 루트에서 실행 — feast·pandas·pyarrow를 설치함
```

> - **4번은 설치가 끝난 뒤에만 실행한다**

- `7-1-feature-store-minimal.py`는 `feast`와 `pyarrow`가 있어야 돌아간다. 없으면 아무것도 만들지 못하고 끝난다.
  - `feast` — 피처 스토어 오픈소스. 피처 정의를 파이썬 파일 하나에 적어 두면 훈련 코드와 서빙 코드가 같은 정의를 읽는다. 3~5절에서 손으로 짠 흐름을 6절에서 이 도구로 다시 돌린다.
  - `pyarrow` — 열 단위 데이터 포맷 Parquet을 읽고 쓰는 라이브러리. Feast로 offline store를 만들면 Parquet 파일이 생기므로 함께 설치한다. 실습 코드에서 직접 호출하지는 않는다.

3. 같은 숫자를 두 곳에서 세면 반드시 어긋난다

□ "지역구별 전일 민원 건수"를 입력으로 쓰는 민원 급증 예측 모델을 생각함
  ○ 피처 스토어가 없으면 이 수치가 두 곳에서 따로 산출됨 — 훈련 파이프라인은 과거 데이터 위에서 집계하고, 서빙 API(serving API — 운영 중에 예측 요청을 받아 예측값을 돌려주는 프로그램)는 요청이 올 때마다 운영 DB에 질의함
  ○ 두 코드는 같은 것을 계산하려는 서로 다른 구현이고, 서로 다른 구현은 어긋남
□ 어긋나는 이유는 실력 차이가 아니라 **정제 단계를 어디까지 적용했는가**의 차이임
  ○ 6주차 정제는 물리 레코드에서 중복을 뺀 뒤 표준 코드로 매핑되지 않은 건을 다시 뺐고, 세 단계 중 어디서 세어도 "그날 민원 건수"라는 이름이 붙음
□ 첫 번째 코드를 실행함. 이 절의 숫자는 전부 이 명령의 출력임

```
cd practice/chapter7
python code/7-2-count-skew.py
```

□ **[2] 정제 단계별 건수** — 같은 날의 민원 건수가 세 값으로 갈림

```
# 출처: practice/chapter7/results/7-2-count-skew.log [2]
[2] 어디까지 정제하고 세느냐에 따라 같은 날의 '민원 건수'가 갈림
    날짜              물리 레코드     중복제거 유효     표준코드 확정
    2026-07-01           20            20            18
    2026-07-02           18            17            16
    2026-07-03           22            22            20
```

  ○ `물리 레코드 / 중복제거 유효 / 표준코드 확정` — 세 열이 6주차 정제의 세 단계임. 오른쪽으로 갈수록 뺀 건이 늘어나는데 어느 열에서 세어도 "그날 민원 건수"라는 같은 이름이 붙음
  ○ `2026-07-02  18  17  16` — 중복 1건이 먼저 빠지고 미매핑 1건이 다시 빠짐. 세 날 모두 양 끝의 차이가 2건이고, 이 절의 나머지 숫자는 이 한 줄에서 나옴
□ **[3] 세 팀이 각자 세면** — 한 날짜에 정답이 세 개가 됨

```
# 출처: 같은 로그 [3]
[3] 세 팀이 각자 '2026-07-02 민원 건수' 피처를 만들면
    서빙 API    운영 DB를 그대로 셈                    18
    분석 부서     중복 1건만 제거                       17
    훈련 파이프라인  중복 + 미매핑 1건까지 제외               16
    최댓값과 최솟값의 차 2건 – 확정 집계 대비 12.5%
    셋 다 버그가 아님: 같은 날의 지역구별 확정 집계는 마포구 5, 관악구 5, 강남구 6 합계 16
```

- ○ `18 / 17 / 16` — 같은 날짜에 정답이 세 개 생김. 어느 것도 계산 오류가 아니고, 차이는 중복 1건과 미매핑 1건을 뺐는지에서만 나옴
- ○ `서빙 API / 분석 부서 / 훈련 파이프라인` — 세는 주체가 셋인 것이 요점임. 조직이 다르면 참조하는 테이블도 정제 단계도 다르게 적용됨
- ○ `차 2건 – 확정 집계 대비 12.5%` — 모델이 16건으로 학습한 뒤 서빙에서 18건을 받으면 학습에 쓴 값보다 12.5% 큰 값이 입력으로 들어감. 모델이 실제로 쓰는 입력은 지역구별 세 값(마포 5·관악 5·강남 6)이고 그 합이 16임

□ **[4] 훈련과 서빙을 맞대어 보면** — 세 날 모두 같은 방향으로 어긋남

```
# 출처: 같은 로그 [4]
[4] 모델이 확정 집계로 훈련되고 서빙은 물리 레코드를 세면
    2026-07-01  훈련  18  서빙  20  차이 +2
    2026-07-02  훈련  16  서빙  18  차이 +2
    2026-07-03  훈련  20  서빙  22  차이 +2
    3일 모두 같은 방향 – 평균 차이 +2.00건
    한쪽으로만 치우친 편의라서 평균 보정으로 사라지지 않음
```

- ○ `차이 +2`가 세 줄 모두 양수임 — 어긋남이 무작위로 흔들리는 것이 아니라 한쪽으로만 치우친 편의이므로, `평균 차이 +2.00건`처럼 평균을 내도 그대로 남음
- ○ `훈련 16  서빙 18` — 모델은 16을 입력으로 "다음 날 급증이 아니었다"를 학습했는데 예측할 때는 18을 받음. 서빙이 정제를 덜 한 쪽이라 입력은 늘 과대 추정되고, 급증이 아닌 날에도 급증으로 예측하는 방향으로 치우침
- ○ 로그 [5]는 이 실행이 `예외 없음, 종료 코드 0`으로 끝났다고 적음 — 어긋난 채로 정상 종료하므로 로그에 남지 않고, 사람이 알아차리는 경로는 모니터링이 아니라 모델 성능 저하뿐임

□ 모델이 16으로 훈련되고 18로 서빙되는 상태를 **훈련-서빙 스큐(training-serving skew)** 라 함

- ○ 1주차가 학습-서빙 불일치라고 부른 것과 같은 현상이고, 두 코드가 오류 없이 계속 도는 탓에 성능 저하의 원인이 몇 주간 모델 쪽에서 탐색됨

□ 피처 스토어의 발상은 계산을 두 번 하지 않는 것임

- ○ 한 번 정의하고, 한 곳에 저장하고, 훈련과 서빙이 같은 곳에서 조회함
- ○ 구체적으로는 세 가지가 일원화됨

- **정의** — 어떤 원천에서 어떤 스키마로 계산되는지를 코드로 선언하고 버전을 붙임(6절)
- **저장** — 값은 피처 스토어에 있고, 훈련용 이력과 서빙용 최신값이 이중으로 유지됨(4절)
- **조회** — 훈련은 시점 조인으로, 서빙은 키 조회로 읽음. API는 달라도 같은 정의에서 나온 값임(5절)

□ 피처 스토어가 16과 18 중 무엇이 옳은지를 판정하지는 않음

○ 하나를 정의로 고르게 하고 모두가 그것만 쓰게 만들 뿐이며, 이 실습의 정의는 새 계산이 아니라 6주차까지 만들어 온 확정 집계임

○ 곧 피처 스토어는 배치 파이프라인이 만든 값을 모델이 읽어 가는 중간 저장소다

□ 반대 논거도 함께 확인함

○ 모델이 하나뿐이고 배치 추론만 하며 피처 계산이 훈련 파이프라인 안에서 끝나면, 훈련과 서빙이 같은 코드에서 나오므로 스큐가 성립하지 않음 — 저장소를 하나 더 들이는 것은 관리 대상만 늘림

○ 이중 구조가 필요해지는 조건은 대체로 셋 중 하나임 — 온라인 서빙이 있어 소비 경로가 실제로 둘인 경우, 여러 모델이 같은 피처를 재사용하는 경우, 미래 누출 감사가 요구되는 경우

4. 저장소를 둘로 나누고 옮겨 담는다 — offline·online·materialize

□ 훈련과 서빙은 같은 피처를 전혀 다르게 소비함

○ 훈련은 "지난 석 달분, 모든 지역구, 모든 날짜"를 한 번에 읽음. 대량이고 과거 이력이 필요하며 몇 분 걸려도 됨

○ 서빙은 "지금, 강남구, 최신값 하나"를 밀리초 안에 읽어야 함. 소량이고 최신값만 있으면 되며 지연을 허용하지 않음

○ 하나의 저장소로 두 패턴을 모두 잘하기 어려워서 저장을 둘로 나눔

□ **offline store**는 피처의 전체 이력을 쌓는 곳임

○ 한 행이 "어느 키의, 어느 시점 값"이고, 날짜가 지나도 이미 쌓인 행을 고치거나 지우지 않고 아래로 더하기만 함 — 그래서 과거 어느 날의 값이든 그대로 남아 있음

○ 그래서 이곳에 필요한 성질은 용량과 대량 읽기임 — 여러 해 분량의 행을 보관하면서 "지난 석 달분을 날짜별로 다 읽어라"를 한 번의 질의로 처리해야 하고, 한 건을 빨리 꺼내는 성질은 필요하지 않음

○ 그 성질에 맞는 저장소가 **데이터 웨어하우스**(BigQuery·Snowflake·Redshift 등)이고, 큰 표를 모아 두고 조건을 걸어 한 번에 읽는 용도로 만든 데이터베이스임. 객체 스토리지에 parquet 파일로 쌓는 방식도 같은 성질을 냄

○ 이 실습은 행이 9개뿐이라 파이썬 리스트와 parquet 파일 하나로 대신함

□ **online store**는 키별 최신값만 두는 곳임

○ 한 키에 한 행만 있고, 적재할 때마다 그 행을 새 값으로 덮어씀 — 이력이 남지 않으므로 "강남구의 7월 2일 값"은 여기서 조회할 수 없음

○ 그래서 이곳에 필요한 성질은 단건 조회 속도임 — 키 하나를 주면 값 하나를 밀리초 안에 돌려줘야 하고, 보관량은 키 개수만큼이라 작음

- ○ 그 성질에 맞는 저장소가 **키-값 저장소**(Redis 등)이고, 키 하나를 받아 거기에 짝지어 둔 값 하나를 메모리에서 바로 꺼내는 데이터베이스임
- ○ 이 실습은 키가 지역구 3개뿐이라 파이썬 사전으로 대신하고, 7-1의 Feast 구성은 Feast 기본값인 SQLite 파일을 씀

□ **materialize**는 오프라인의 값을 온라인으로 옮겨 담는 배치 작업임

- ○ 두 저장소가 따로 있으므로 온라인 쪽이 저절로 채워지지 않음 — 누군가 주기적으로 복사해 주지 않으면 서빙은 빈 저장소를 조회함
- ○ 하는 일은 두 단계임 — 먼저 적재할 시간 구간(창)을 받아 그 안에 든 행만 고르고, 그다음 키마다 타임스탬프가 가장 늦은 행 하나만 남겨 온라인에 덮어씀
- ○ 그래서 이력 전체가 복사되지 않고 키 개수만큼의 행만 남으며, 적재 주기가 곧 서빙이 쓰는 값이 최대 얼마나 묵은 것일 수 있는지를 정함 — 운영에서는 cron·Airflow 같은 스케줄러가 정해진 주기로 이 작업을 호출함
- ○ 창 안에 행이 하나도 없어도 오류가 아니라 0행을 옮기고 정상 종료함. 이 성질이 뒤의 [4]에서 문제가 됨

□ 두 번째 코드를 실행함. 일별 집계를 피처 행으로 펴고, 적재하고, 두 저장소를 대조함

```
python code/7-3-materialize-consistency.py
```

□ **[1] offline store** — 날짜별 행이 쌓임

```
# 출처: practice/chapter7/results/7-3-materialize-consistency.log [1]
[1] offline store – 피처 이력 전체 9행 (3일 × 지역구 3개)
    lawd_cd   지역      알려진 시점                    건수      미매핑율
    11440     마포구     2026-07-02T00:00:00Z         5     0.050
    11620     관악구     2026-07-02T00:00:00Z         5     0.050
    11680     강남구     2026-07-02T00:00:00Z         8     0.050
    11440     마포구     2026-07-03T00:00:00Z         5     0.059
    11620     관악구     2026-07-03T00:00:00Z         5     0.059
    11680     강남구     2026-07-03T00:00:00Z         6     0.059
    11440     마포구     2026-07-04T00:00:00Z         6     0.045
    11620     관악구     2026-07-04T00:00:00Z         5     0.045
    11680     강남구     2026-07-04T00:00:00Z         9     0.045
```

- ○ `9행 (3일 × 지역구 3개)` — 한 행이 "그 구의 그날 확정 집계" 하나를 담음. `알려진 시점`은 대상일이 아니라 값이 확정된 시각이고 대상일 다음 날 자정으로 적으므로(`7-3-materialize-consistency.py:45`), `2026-07-04` 행은 7월 3일분 집계임
- ○ `11680 강남구`의 `8 → 6 → 9` — 같은 키의 값이 날마다 달라지므로 훈련은 이 세 행을 전부 읽어야 변동을 학습함. 미매핑율이 건수와 같은 행에 들어간 것은 피처 값과 그 값의 품질 지표를 같은 시점에 묶어 두기 위함임

□ **[2] materialize** — 키별 최신 1행만 옮겨 담음

```
# 출처: 같은 로그 [2]
[2] materialize – 적재 창을 명시해서 호출
    창 [2026-07-01T00:00:00Z, 2026-07-08T00:00:00Z]
    창 안의 원천 9행 → online store 3행(키별 최신 1행)
    11440  마포구      6  0.045  @2026-07-04T00:00:00Z
    11620  관악구      5  0.045  @2026-07-04T00:00:00Z
    11680  강남구      9  0.045  @2026-07-04T00:00:00Z
```

- ○ `창 안의 원천 9행 → online store 3행` — 예측표에 9를 적었다면 두 저장소의 성격 차이를 놓친 것임. 고르는 기준은 "학습에 쓰였는가"가 아니라 "그 키에서 타임스탬프가 가장 늦은 행인가"여서 마지막 날의 마포 6·관악 5·강남 9가 남음
- ○ 옮겨 담은 값에 타임스탬프가 함께 붙으므로 서빙이 쓰는 값이 며칠 전 것인지 셀 수 있음. 적재할 구간(창)은 호출할 때 명시하며, 이 창이 무엇을 결정하는지는 [4]에서 드러남

□ **[3] 일관성 점검** — 두 저장소가 같은 값을 들고 있는지 대조함

```
# 출처: 같은 로그 [3]
[3] 훈련-서빙 일관성 점검 – 오프라인 최신값 대 온라인 조회값
    키       오프라인 최신          온라인 조회          일치
    11440   6 / 0.045          6 / 0.045          예
    11620   5 / 0.045          5 / 0.045          예
    11680   9 / 0.045          9 / 0.045          예
    checked_keys 3, all_consistent True
```

- ○ `all_consistent True` — 이 표가 침묵 실패를 값으로 드러내는 점검이고, 없으면 서빙이 `None`을 받는 상태가 오류 없이 지나감. 자명해 보이는 일치는 두 저장소를 같은 정의에서 적재했기 때문에 온 것이지 저절로 오지 않음
- ○ `checked_keys 3` — 점검 대상이 0개이면 `all_consistent`는 True로 나오므로(`7-3-materialize-consistency.py:89`) 두 값을 같이 읽어야 함
- ○ 저장된 값이 같다고 안심할 수는 없음 — 서빙 코드가 값을 읽은 뒤 다르게 변형하면 모델이 받는 입력은 여전히 어긋나므로, 운영에서는 모델로 들어간 값을 로그로 남겨 훈련 분포와 정기 비교함 (11주차)

□ **[4] 적재 창을 잘못 잡으면** — 0행을 옮기고도 성공으로 끝남

```
# 출처: 같은 로그 [4]
[4] 같은 적재를 증분 호출로 하면 – 시작점이 '기준 시각 - TTL'
    기준 시각 2026-07-08T00:00:00Z(구성값), TTL 3일
    창 [2026-07-05T00:00:00Z, 2026-07-08T00:00:00Z]
    창 안의 원천 0행 → online store 0행
    예외 없음, 오류 로그 없음 – 적재 작업은 성공으로 끝남
    이 상태에서 온라인을 조회하면
    키       오프라인 최신          온라인 조회          일치
    11440   6 / 0.045          없음(None)          아니오
    11620   5 / 0.045          없음(None)          아니오
    11680   9 / 0.045          없음(None)          아니오
    checked_keys 3, all_consistent False
```

- ○ `TTL 3일`은 피처를 며칠까지 유효로 볼지 미리 정해 둔 값임(6절). 기준 시각은 마지막 피처가 확정된 7월 4일에서 나흘 뒤인 7월 8일이라 창이 7월 5~8일이 되고, 실습 피처(7월 2~4일)는 창 바깥이라 `창 안의 원천 0행`임 — 일별 배치가 나흘 멈춘 상태에 해당함
- ○ `예외 없음, 오류 로그 없음` — 3행을 옮긴 [2]와 0행을 옮긴 여기를 종료 코드로는 구별할 수 없으므로, 적재 건수와 `all_consistent`를 따로 봐야 드러남. 4주차 유실·5주차 폐기·6주차 정제 drop에 이어 네 번째로 같은 구조임
- ○ `없음(None)` 세 줄 — 서빙이 피처를 받지 못하는 상태임. "없으면 0으로" 같은 기본값을 서빙 코드에 두면 모든 지역구가 0인 채로 서빙되므로, 결측은 덮지 말고 **피처 결측률**로 세어 둠(11주차)

□ 적재 주기를 정하는 순서는 거꾸로임 — 먼저 모델이 견딜 수 있는 피처의 낡음을 정하고 주기를 그 안쪽으로 잡음

- ○ 낡음의 정도를 **피처 신선도**로 재는데, 온라인 스토어에 있는 값의 타임스탬프와 현재 시각의 차이임. 하루 한 번 적재하면 최댓값이 약 24시간이 됨
- ○ 24시간이 괜찮은지는 피처가 가리키는 현상이 얼마나 빨리 변하는가와, 그 값으로 내린 결정을 되돌리는 비용이 얼마인가가 정함
- ○ "전일 민원 건수"로 다음 날 인력 배치를 정한다면 하루 지연은 정의상 문제가 아니지만, 재난 신고 밀집도로 출동 순서를 정한다면 24시간 전 값은 쓸 수 없고 잘못 정한 순서를 되돌릴 방법도 없음

□ 같은 흐름을 Feast 엔진으로 실행한 결과도 같았음 — `all_consistent` true, `checked_keys` 3, 키별 값은 마포 6·관악 5·강남 9, 미매핑율은 세 곳 모두 0.045임
(`practice/chapter7/data/output/ch7_consistency_report.json`, 과거 Feast 실행 산출물)

5. 그 시점에 알 수 있었던 값만 붙인다 — 시점 조인

□ 훈련 샘플 하나는 "7월 2일 오전 9시의 강남구"처럼 키와 시점의 쌍이고, 여기에 "전일 민원 건수"를 붙여야 함

- ○ 7월 2일 하루 전체의 집계를 붙이면 그 값은 7월 2일 오전 9시에 아직 존재하지 않던 값임. 그날이 끝나야 확정되기 때문임
- ○ 이렇게 그 시점에 확정되지 않은 값이 과거 시점의 샘플에 들어간 것을 **미래 누출(future leakage)**이라 함. 훈련과 평가에서 성능이 비현실적으로 좋게 나오다가 서빙에서 떨어지므로, 성능이 좋아 보이는 것 자체가 증상임

□ **시점 조인(point-in-time join)** 이 이것을 조회 규칙으로 막음

- ○ 각 샘플의 기준 시점마다 **그 시점 이전에 알려진 가장 최근 값만** 붙임
- ○ 그래서 피처의 타임스탬프는 "값이 가리키는 기간"이 아니라 "값이 알려진 시점"이어야 함 — 실습이 7월 1일분 집계에 7/1이 아니라 **7/2 00:00**을 붙인 이유임
- ○ 2주차의 이벤트 시간과 처리 시간, 5주차의 워터마크, 6주차의 logical date에 이어 네 번째로 확인하는 시간 구분임. 이번에는 "알 수 있게 된 시각"임

□ 세 번째 코드를 실행함. 기준 시점이 다른 훈련 샘플 4개에 시점 조인을 적용함

```
python code/7-4-point-in-time-join.py
```

□ **[2] 시점 조인** — 기준 시점 이전에 알려진 값만 붙음

```
# 출처: practice/chapter7/results/7-4-point-in-time-join.log [2]
[2] 시점 조인 – 훈련 샘플 4행
    11680  as_of 2026-07-02T09:00:00Z  →  건수 8, 미매핑율 0.050
           쓴 피처: 2026-07-01분, 알려진 시점 2026-07-02T00:00:00Z (간격 0일 9시간)
    11440  as_of 2026-07-03T09:00:00Z  →  건수 5, 미매핑율 0.059
           쓴 피처: 2026-07-02분, 알려진 시점 2026-07-03T00:00:00Z (간격 0일 9시간)
    11680  as_of 2026-07-04T09:00:00Z  →  건수 9, 미매핑율 0.045
           쓴 피처: 2026-07-03분, 알려진 시점 2026-07-04T00:00:00Z (간격 0일 9시간)
    입력 4행 → 결과 3행 (탈락 1행)
    탈락: 11620  as_of 2026-07-01T09:00:00Z  – 확정된 값 없음
    탈락 행은 빈 값으로 남지 않고 결과에서 사라짐 – 오류도 경고도 없음
```

- ○ `11680`이 두 줄에 나오고 `8`과 `9`가 각각 붙음 — 같은 키라도 기준 시점이 다르면 그때 알 수 있었던 값이 다름. 이것이 시점 조인의 정의임
- ○ `쓴 피처: 2026-07-01분, 알려진 시점 2026-07-02T00:00:00Z` — 붙인 값이 어느 날짜분이고 언제 확정됐는지를 함께 남김. 이 두 칸이 없으면 누출이 있었는지 나중에 확인할 방법이 없음
- ○ `입력 4행 → 결과 3행 (탈락 1행)` — 7월 1일 오전 9시에는 확정된 집계가 하나도 없어 붙일 값이 없음. 그 샘플은 빈 값으로 남는 것이 아니라 결과에서 사라지고, `오류도 경고도 없음`

□ 시점 조인에는 하한도 있음 — 알려진 값이어도 오래되면 붙이지 않음(같은 로그 [4])

- ○ 7월 10일 샘플에는 7월 4일에 확정된 값이 있어 미래도 아니지만, 6일 9시간 전이라 TTL 3일을 넘겨 탈락함. 상한(미래 금지)과 하한(TTL)을 둘 다 통과해야 피처가 붙음

- ○ 하한을 두는 이유는 석 달 전의 "전일 민원 건수"가 알려진 값이긴 해도 "전일"이라는 피처의 의미를 잃기 때문임
- ○ 그래서 TTL은 디스크나 성능이 아니라 **배치 실패를 며칠까지 견딜 것인가**에서 역산함 — 실습의 요구사항이 "하루 실패한 적은 있고 이틀 연속은 없다"였으므로 3일로 잡았고, 1일로 잡으면 하루 실패한 다음 날부터 훈련 샘플이 통째로 빠짐
- ○ 배치가 TTL보다 오래 멈추면 훈련 샘플이 이렇게 조용히 사라짐 — 4절 [4]의 0행 적재와 같은 구조이고, 서빙이 `None`을 받는 대신 훈련 샘플이 줄어듦

□ 기준 시점은 운영에서 실제로 예측이 호출되는 시각과 같아야 함

- ○ 실습이 각 날짜의 09:00을 쓴 것은 "오전 9시에 그날의 인력 배치를 정한다"는 업무 시나리오에서 나온 구성값임
- ○ 09:00에 결정하면서 훈련 샘플의 기준 시점을 23:59로 두면 그날 낮에 확정된 값이 붙어 설정값 하나로 미래 누출이 들어옴

□ 같은 조인을 Feast 엔진으로 실행한 결과도 같은 값을 남겼음

- ○ 입력 4행에 조인 결과 3행, 탈락 1행이고 붙은 값은 8·5·9임 (`practice/chapter7/data/output/ch7_feature_report.json`, 과거 Feast 실행 산출물)

6. 피처 정의서 — 누가 책임지고 언제 알려진 값인가

□ 피처가 한 곳에 정의되고 여러 모델이 재사용하는 자산이 되는 순간, 기술 문제와 함께 책임 문제가 옴

- ○ 원천 부서가 집계 방식을 바꿨는데 소비하는 모델 셋이 그것을 모르면 사고가 남
- ○ 그래서 정의서에는 계산 명세와 함께 네 가지가 들어감 — **소유자**(정의와 품질에 책임지고 변경을 알릴 부서), **원천과 갱신 주기**, **개인정보 등급**, **타임스탬프 의미**
- ○ 개인정보 등급은 피처 단위로 붙음. 지역 단위 집계는 비식별이지만 가구·개인 단위 피처에는 접근 통제와 이용 목적 제한이 붙으므로, "피처 스토어에 접근 가능"이라는 시스템 단위 권한은 뜻이 없음

□ 네 번째 코드를 실행함. 이 절의 값은 과거 실행이 남긴 산출물에서 옮긴 것임

```
python scripts/setup_practice.py 7        # 저장소 루트에서, 먼저 설치
cd practice/chapter7 && python code/7-1-feature-store-minimal.py
```

□ 정의서는 사람이 쓴 문서가 아니라 등록된 정의를 registry에서 그대로 뽑은 JSON임 — 코드와 문서가 어긋날 수 없는 이유가 여기 있음(`practice/chapter7/data/output/ch7_feature_definitions.json`)

- ○ 조회 키는 `"join_key": "lawd_cd"`로 선언되어 지역명이 아니라 코드임. 6주차의 "이름이 아니라 코드로 붙인다"는 표준 코드 원칙이 그대로 반복됨
- ○ 등록된 피처는 `complaint_count`와 `day_unmapped_rate` 둘이고, 값과 그 값을 만든 정제의 품질을 함께 등록함 — 4절 로그에서 미매핑율이 건수와 같은 행에 있던 이유임
- ○ 각 피처에 타입과 원천 산출물 이름이 함께 적혀 있어, 값이 이상할 때 거슬러 올라갈 파일이 정의서 안에 있음

□ **governance** — 소유와 책임을 피처 단위로 적음

```
# 출처: 같은 JSON의 feature_view.governance
    "governance": {
      "owner": "민원데이터팀(가상 – 시뮬레이션 표기)",
      "privacy": "개인정보 없음(지역 단위 집계)",
      "source": "7장 일별 배치 확정 산출물",
      "update_cycle": "일 1회(배치 마감 후)"
    },
```

○ `"owner"`가 `(가상 – 시뮬레이션 표기)`로 적혀 있음 — 실제 운영에서는 연락 가능한 부서가 들어가야 이 칸이 뜻을 가짐

○ `"update_cycle": "일 1회(배치 마감 후)"` — 갱신 주기를 적어 두므로, 온라인 값이 하루 낡을 수 있다는 사실이 소비자에게 미리 전달됨

○ `"source"`에 적힌 "7장"은 참고 원고의 장 번호이고, 강의 주차로는 6주차의 일별 배치 산출물을 가리킴

□ **타임스탬프 규약과 TTL** — 5절의 규칙을 문서에 값으로 남김

```
# 출처: 같은 JSON의 feature_view 나머지
    "name": "complaint_daily",
    "timestamp_semantics": "event_timestamp = 값이 확정·공개된 시점(집계 대상일 다음 날 00:00 UTC)",
    "ttl_days": 3
  },
  "project": "public_complaints"
```

○ `"timestamp_semantics"` — 5절의 "알려진 시점" 규약을 한 문장으로 적어 둠. 미래 누출 감사에 답할 때 근거가 되는 문장이고, 4절 로그의 `@2026-07-04` 행이 7월 3일분이었던 이유도 이 한 줄에 적혀 있음

○ `"ttl_days": 3` — 5절에서 배치 실패 견딤으로 역산한 3일이 문서에도 값으로 남음. 조인 결과가 이상할 때 먼저 확인할 숫자임

□ 무엇을 키로 삼을지는 **서빙 시점에 무엇을 조회하는가**가 정함

○ 상황실 담당자가 "강남구의 현재 상태"를 조회하면 키는 지역구이고, 민원 접수 담당자가 "이 민원 건의 예상 처리 시간"을 조회하면 키는 민원 건 ID여야 함

○ 단위를 잘게 잡으면 피처는 정밀해지지만 온라인 스토어의 키 개수가 늘고, 키마다 이력이 짧아져 5절의 행 탈락도 늘어남

□ 정의는 불변이 아님

- ○ 정제 규칙이 바뀌면 같은 이름의 피처가 다른 값을 갖게 되고, 기존 정의를 알림 없이 덮어쓰면 그 피처로 훈련된 모델들의 훈련-서빙 일치가 소급으로 무효가 됨 — 스큐를 막으려던 구조가 스큐의 원인이 되는 역설임
- ○ 그래서 정의 변경은 새 feature service 버전으로 내고 기존 버전은 소비 모델이 이전을 마칠 때까지 유지함. `complaint_model_v1`의 v1이 그 규율을 적어 둔 자리임

□ 이 정의서는 다음 주차의 입력으로 쓰임

- ○ 9주차는 이 피처로 만든 학습 데이터의 지문 해시(데이터 전체를 한 값으로 요약한 해시)를 실험 기록에 남기고, 13주차 검수표는 통합 시스템이 처리한 데이터의 지문을 그 값과 대조함
- ○ 두 값이 일치해야 "9주차에 학습에 쓴 데이터"와 "운영이 처리한 데이터"가 같다고 말할 수 있다

7. 핵심 요약

□ **1. 피처를 두 곳에서 따로 계산하면 반드시 어긋나고, 그 어긋남은 오류로 드러나지 않는다.** (3절)

- ○ 7월 2일 민원 건수는 어디까지 정제했느냐에 따라 18·17·16으로 갈렸고 셋 다 버그가 아니었음
- ○ 세 날 모두 차이가 +2로 같은 방향이라 평균 보정으로 사라지지 않고, 프로그램은 종료 코드 0으로 끝났음
- ○ 피처 스토어는 무엇이 옳은지 판정하지 않고 하나를 정의로 고르게 한 뒤 훈련과 서빙이 그것만 읽게 만듦

□ **2. 소비 패턴이 달라서 저장소를 둘로 나누고, 적재는 건수로 확인한다.** (4절)

- ○ 이력 9행이 offline store에 있고 materialize가 옮긴 최신 3행이 online store에 남았음
- ○ 적재 창을 증분 기본값으로 바꾸자 0행이 적재됐는데 예외도 오류 로그도 없었음
- ○ 그래서 적재는 성공 로그가 아니라 `창 안의 원천 N행`으로 확인함

□ **3. 두 저장소의 값이 같다는 것은 대조해서 보이는 것이다.** (4절)

- ○ 세 지역구에서 오프라인 최신값과 온라인 조회값을 비교해 `checked_keys` 3, `all_consistent` True를 얻었음
- ○ 같은 점검을 0행 적재 상태에 돌리면 세 키 모두 불일치로 잡히므로, 이 표가 침묵 실패를 값으로 드러내는 점검임
- ○ 자명해 보이는 일치는 같은 정의에서 적재했기 때문에 온 것이지 저절로 오지 않음

□ **4. 훈련 데이터에는 그 시점에 알 수 있었던 값만 붙인다.** (5절)

- ○ 타임스탬프를 "값이 알려진 시점"으로 두고 기준 시점 이전의 최신 값만 조인하니 강남구 두 샘플에 8과 9가 붙었음
- ○ 기준 시점 조건을 빼고 조회하면 그 시점에 확정되지 않은 값까지 붙어 훈련 셋이 줄지 않으므로, 샘플이 더 많이 남는 쪽이 오히려 누출된 쪽임
- ○ 조인할 값이 없는 샘플은 행째 사라지고 TTL 3일을 넘긴 값도 붙지 않으므로, 조인 전후 행 수 대조는 선택이 아니다

□ **5. 정의서는 피처가 자산이 되는 순간 필요해진다.** (6절)

- ○ 소유자·원천·갱신 주기·개인정보 등급·타임스탬프 의미를 registry에서 뽑아 문서로 남겼음

○ 정의가 바뀌면 덮어쓰지 않고 새 버전으로 내며, `complaint_model_v1`의 v1이 그 자리 표시임

○ 이 문서와 9주차의 지문 해시가 함께 있어야 "무엇으로 학습했는가"에 답할 수 있다

8. 과제

□ 3~5절의 코드를 순서대로 실행해 만든 로그 세 개를 LMS에 제출한다

1. `practice/chapter7/results/7-2-count-skew.log`
2. `practice/chapter7/results/7-3-materialize-consistency.log`
3. `practice/chapter7/results/7-4-point-in-time-join.log`

□ 이어서 네 가지를 각각 세 문장 이내로 쓴다

1. 본인 로그 `7-2`의 [3]에 나온 세 값(18·17·16)을 옮겨 적고, 셋 다 계산 오류가 아닌 이유를 밝힌다. 이어서 어느 값을 피처 정의로 고를지와 그 근거를 쓴다.
2. 본인 로그 `7-3`의 [3]에서 `checked_keys`와 `all_consistent`, 그리고 키 하나의 오프라인·온라인 값을 옮겨 적는다.
3. 6주차의 일별 배치가 **나흘 연속 실패**했다고 하자. 나흘째에 온라인 스토어를 조회하면 무엇이 반환되고, 같은 시점에 오프라인 훈련 셋을 만들면 어떤 문제가 생기는가. 본인 로그 `7-3`의 [4] 블록과 TTL 3일을 근거로 답한다.
4. 이 사고를 배치 실패 직후에 잡으려면 어느 지점에 무엇을 두어야 하는가. 감지 지점 두 개를 제시하고 각각이 무엇을 신호로 삼는지 쓴다.

□ 값을 지어내지 않는다. 실행이 안 되면 어디서 막혔는지 그대로 쓴다

□ 수업일 포함 7일 이내에 제출한다